软件工程（Software Engineering）

一、学科体系与核心概念

- 1. 定义与学科定位
  - 定义（IEEE 610.12）：将系统化、规范化、可度量的方法应用于软件的开发、运行与维护
  - 目标：在成本、进度、质量三角中取得平衡，交付满足需求的软件
  - 与计算机科学的区别：CS 研究「能不能算」，SE 研究「如何可靠地造出来」
  - 软件危机（1968 NATO 会议）：进度失控、成本超支、质量不可控，软件工程由此确立
  - 软件的本质特征（Brooks）
    - 复杂性（Complexity）：实体间依赖错综复杂
    - 一致性（Conformity）：需适配既有接口与规范
    - 可变性（Changeability）：需求与环境持续变化
    - 不可见性（Invisibility）：无物理形态，难以直观度量
  - 第一性原理：软件是逻辑产品，其成本主要在人力与沟通，而非复制
- 2. 软件生命周期（SDLC）
  - 可行性分析：技术、经济、操作、法律可行性
  - 需求分析：功能需求、非功能需求、约束
  - 系统设计：概要设计、详细设计
  - 编码实现
  - 测试验证
  - 部署交付
  - 运行维护：纠错性、适应性、完善性、预防性维护
  - 退役与迁移
  - 阶段交付物：可行性报告、SRS、设计说明书、源码、测试报告、运维手册
- 3. 过程模型（Process Models）
  - 瀑布模型（Waterfall）：阶段串行、文档驱动，适合需求稳定且变更少的项目
  - V 模型：开发阶段与测试阶段一一对应，强调验证与确认
  - 增量模型（Incremental）：分批交付，每批可用
  - 迭代模型（Iterative）：循环细化，逐步逼近
  - 螺旋模型（Spiral）：风险驱动，每圈含风险分析，适合大型高风险项目
  - 原型模型（Prototype）：快速验证需求，适合需求不明确
  - RUP（统一过程）：用例驱动、架构为中心、迭代增量；四阶段（初始 / 细化 / 构造 / 交付）
  - 敏捷（Agile）
    - 敏捷宣言四价值观：个体与交互、可工作的软件、客户协作、响应变化
    - 十二条原则：短周期交付、拥抱变化、自组织团队、持续改进
  - Scrum
    - 三角色：产品负责人 PO、敏捷教练 SM、开发团队
    - 三工件：产品待办列表、Sprint 待办列表、增量
    - 五会议：Sprint 计划、每日站会、评审会、回顾会、待办梳理
    - 估算：故事点、计划扑克、燃尽图
  - XP（极限编程）：结对编程、测试驱动开发、持续集成、小版本发布、简单设计
  - Kanban：看板、在制品限制 WIP、拉动式流转、度量周期时间
  - DevOps：开发运维一体化、CI/CD、基础设施即代码、文化变革
  - 能力成熟度 CMMI：初始级、已管理级、已定义级、量化管理级、优化级
- 4. 方法论演进
  - 结构化方法（1970s）：自顶向下、功能分解、数据流图 DFD、数据字典
  - 面向对象方法（1980s–1990s）：封装、继承、多态；UML 统一建模
  - 构件化开发（CBD）：构件复用、组装、接口契约
  - 面向服务架构 SOA：服务编排、企业服务总线 ESB
  - 微服务架构（2014+）：单一职责、独立部署、去中心化数据管理
  - 云原生：容器化、服务网格、声明式 API、Serverless
  - 平台工程（Platform Engineering）：内部开发者平台 IDP、自助式交付
  - AI 辅助开发：代码补全、单测生成、需求分析、代码评审
- 5. 软件类型与特征
  - 系统软件：操作系统、数据库、编译器
  - 应用软件：企业应用、办公、行业软件
  - 嵌入式与实时软件：强时序约束、资源受限
  - 分布式与云原生软件：弹性、多租户
  - 安全攸关软件：航空、医疗、核电，需形式化验证
  - SaaS / PaaS / IaaS 形态差异

二、方法与技术体系

- 1. 需求工程（Requirements Engineering）
  - 需求分类
    - 功能需求（Functional）：系统应做什么
    - 非功能需求（Non-Functional）：性能、安全、可用性、可维护性
    - 约束（Constraint）：技术栈、法规、预算
  - 需求获取：访谈、问卷、观察、原型、文档分析、头脑风暴、联合应用开发 JAD
  - 需求分析
    - 用例建模（Use Case）：参与者、场景、主流程与异常流
    - 用户故事（User Story）：As a... I want... so that...；INVEST 原则
    - 场景分析、数据流分析
  - 需求规格说明 SRS：IEEE 830 结构，可验证、无歧义、可追踪
  - 需求验证：评审、原型确认、可测试性检查、一致性检查
  - 需求管理
    - 需求基线、变更控制委员会 CCB
    - 需求追踪矩阵 RTM：需求 → 设计 → 代码 → 测试
  - 优先级方法：MoSCoW（必须有/应该有/可以有/不会有）、Kano 模型、价值-成本矩阵
  - 常见问题：需求蔓延（Scope Creep）、模糊需求、隐含需求、伪需求
- 2. 软件设计（Software Design）
  - 设计原则
    - 模块化、抽象、信息隐藏、逐步求精
    - 高内聚、低耦合
    - SOLID：单一职责 SRP、开闭 OCP、里氏替换 LSP、接口隔离 ISP、依赖倒置 DIP
    - 其他：DRY、KISS、YAGNI、迪米特法则 LoD、组合优于继承
  - 结构化设计
    - 模块结构图、调用关系
    - 耦合七级：非直接 → 数据 → 标记 → 控制 → 外部 → 公共 → 内容
    - 内聚七级：功能 → 顺序 → 通信 → 过程 → 时间 → 逻辑 → 偶然
  - 面向对象设计
    - 类与对象、职责分配 GRASP 模式
    - 领域模型、贫血模型 vs 充血模型
    - 聚合与限界上下文（DDD）
  - 设计模式（GoF 23 种）
    - 创建型 5：单例、工厂方法、抽象工厂、建造者、原型
    - 结构型 7：适配器、桥接、组合、装饰器、外观、享元、代理
    - 行为型 11：责任链、命令、解释器、迭代器、中介者、备忘录、观察者、状态、策略、模板方法、访问者
  - UML 建模
    - 结构图：类图、对象图、包图、组件图、部署图
    - 行为图：用例图、活动图、状态机图、时序图、通信图
    - 关系：关联、聚合、组合、泛化、实现、依赖
  - 架构设计
    - 架构风格：分层、管道-过滤器、事件驱动、微内核、CQRS、事件溯源
    - 应用架构模式：MVC、MVP、MVVM、六边形架构、洋葱架构、整洁架构
    - 分布式架构：SOA、微服务、服务网格、Serverless、Actor 模型
    - 架构质量属性：性能、可用性、可修改性、安全性、可测试性、可扩展性
    - 架构评估：ATAM、SAAM、场景走查
    - 架构决策记录 ADR
- 3. 编码与实现
  - 编程范式：面向过程、面向对象、函数式、响应式、并发模型（线程 / 协程 / Actor）
  - 编码规范：命名约定、注释规范、格式化、异常处理、日志规范
  - 代码质量
    - 可读性、可维护性、圈复杂度、重复率、函数长度
    - 坏味道识别：长函数、大类、重复代码、过度耦合、霰弹式修改
  - 重构（Refactoring）：提取方法、内联、搬移、重命名、以查询取代临时变量
  - 代码评审：Pull Request 流程、检查清单、结对评审
  - 静态分析：SonarQube、ESLint、Pylint、Checkstyle、FindBugs
  - 版本控制
    - Git 核心：分支、合并、变基、cherry-pick、冲突解决
    - 工作流：GitFlow、GitHub Flow、Trunk-Based Development
    - 提交规范：Conventional Commits、语义化版本 SemVer
  - 依赖管理：包管理器、锁文件、依赖冲突、漏洞扫描（SCA）
  - 技术文档：API 文档（OpenAPI）、架构文档、README、变更日志
- 4. 软件测试（Software Testing）
  - 测试层次：单元测试 → 集成测试 → 系统测试 → 验收测试
  - 测试类型：功能、性能、安全、兼容性、易用性、回归、冒烟、探索式测试
  - 黑盒方法：等价类划分、边界值分析、判定表、因果图、正交实验、错误推测
  - 白盒方法：语句覆盖、判定覆盖、条件覆盖、判定-条件覆盖、路径覆盖、MC/DC
  - 测试驱动开发 TDD：红 - 绿 - 重构
  - 行为驱动开发 BDD：Given-When-Then、Cucumber、SpecFlow
  - 自动化测试
    - 单元：JUnit、Pytest、TestNG
    - 接口：Postman、RestAssured
    - UI：Selenium、Playwright、Cypress、Appium
    - 性能：JMeter、Locust、LoadRunner、Gatling
  - 性能指标：TPS、响应时间 RT、并发数、资源占用、P99 延迟
  - 安全测试：SAST、DAST、IAST、渗透测试、模糊测试 Fuzzing
  - 缺陷管理：缺陷生命周期、严重程度与优先级、缺陷密度、逃逸率
  - 测试策略：测试金字塔（单元 > 接口 > UI）、契约测试、混沌工程
- 5. 运维与交付（DevOps）
  - 持续集成 CI：自动构建、自动测试、代码质量门禁
  - 持续交付/部署 CD：流水线、制品库、环境推进
  - 工具链：Jenkins、GitLab CI、GitHub Actions、Argo CD、Tekton
  - 容器化
    - Docker：镜像分层、Dockerfile 多阶段构建、卷与网络
    - 镜像仓库：Harbor、ECR、ACR
  - 容器编排 Kubernetes
    - 核心对象：Pod、Deployment、Service、Ingress、ConfigMap、Secret
    - 调度与弹性：HPA、亲和性、资源请求与限制
    - 生态：Helm、Operator、Service Mesh（Istio / Linkerd）
  - 基础设施即代码 IaC：Terraform、Ansible、Pulumi、CloudFormation
  - 配置与环境：配置中心（Nacos / Apollo）、多环境隔离
  - 可观测性
    - 指标：Prometheus + Grafana
    - 日志：ELK / EFK、Loki
    - 链路追踪：Jaeger、SkyWalking、OpenTelemetry
  - 发布策略：蓝绿部署、金丝雀发布、灰度、滚动更新、特性开关 Feature Flag
  - 可靠性工程 SRE：SLI / SLO / SLA、错误预算、故障复盘 Postmortem
  - 混沌工程：Chaos Mesh、故障注入
- 6. 项目管理
  - 知识体系：PMBOK 十大知识领域、五大过程组
  - 范围管理：WBS 工作分解结构、范围蔓延控制
  - 进度管理：甘特图、关键路径法 CPM、PERT 三点估算、里程碑
  - 成本管理：挣值分析 EVM（PV / EV / AC、CPI / SPI）
  - 质量管理：质量保证 QA 与质量控制 QC 的区别
  - 风险管理：风险识别、概率-影响矩阵、应对策略（规避 / 转移 / 减轻 / 接受）
  - 人力资源管理：塔克曼团队发展阶段（形成 / 震荡 / 规范 / 执行 / 解散）、布鲁克斯定律
  - 沟通管理：干系人分析、沟通计划、会议机制
  - 配置管理：配置项识别、基线、变更控制、审计
  - 度量管理：交付周期、吞吐量、在制品、缺陷趋势
- 7. 数据库与数据访问设计
  - 关系模型：实体、属性、关系、完整性约束
  - 范式：1NF、2NF、3NF、BCNF；反范式化的性能权衡
  - 键与约束：主键、外键、唯一约束、检查约束、级联策略
  - 索引：B+ 树原理、哈希索引、联合索引最左前缀、覆盖索引、索引失效场景
  - 事务：ACID、隔离级别（读未提交 / 读已提交 / 可重复读 / 串行化）、MVCC、幻读
  - 锁机制：行锁、表锁、间隙锁、意向锁、死锁检测与规避
  - 查询优化：执行计划 EXPLAIN、慢查询日志、分页深翻优化
  - 分库分表：垂直拆分、水平拆分、路由策略、跨库 JOIN、全局 ID
  - 分布式事务：2PC、TCC、Saga、本地消息表、最大努力通知
  - 数据访问层：ORM、SQL 构建器、连接池、读写分离
- 8. 并发与性能工程
  - 并发模型：多线程、线程池、协程、Reactor、Actor、事件循环
  - 同步原语：互斥锁、读写锁、CAS、原子类、内存屏障、AQS
  - 缓存
    - 层次：本地缓存（Caffeine）、分布式缓存（Redis）
    - 问题：缓存穿透、击穿、雪崩、一致性、热点 Key
  - 消息队列：削峰填谷、系统解耦、异步化、幂等消费、顺序保证、死信队列
  - 流量治理：限流（令牌桶、漏桶、滑动窗口）、熔断、降级、隔离舱
  - 性能分析方法：压测、火焰图、链路追踪、GC 日志、慢 SQL 分析
  - 关键指标：QPS、RT、P99、吞吐量、资源利用率、并发连接数
  - 容量规划：单机压测 → 线性外推 → 预留冗余

三、质量、度量与评估

- 1. 质量模型
  - ISO/IEC 25010 八大特性
    - 功能适合性、性能效率、兼容性、易用性
    - 可靠性、安全性、可维护性、可移植性
  - McCall 模型：产品运行、产品修正、产品转移
  - 质量成本：预防成本、评估成本、内部失败成本、外部失败成本
  - 质量内建（Quality Built-in）：把质量前移到需求与设计阶段
- 2. 软件度量
  - 规模度量：代码行 LOC、功能点 FP、故事点、用例点
  - 复杂度度量：圈复杂度 McCabe、Halstead 度量、认知复杂度
  - 耦合与内聚度量
  - 缺陷度量：缺陷密度、缺陷发现率、缺陷重开率、缺陷逃逸率
  - 可靠性度量：MTBF、MTTR、MTTF、可用性 = MTBF / (MTBF + MTTR)
  - 过程度量：交付周期时间、吞吐量、在制品、流动效率
  - 技术债：本金与利息、技术债比率、偿还策略
  - 度量陷阱：古德哈特定律、指标被游戏化
- 3. 评估方法
  - 代码评审与走查
  - 静态分析与质量门禁
  - 测试覆盖率（语句 / 分支 / 路径）与变异测试
  - 可用性测试、A/B 测试
  - 架构评估：ATAM、SAAM、质量属性场景
  - 性能压测与容量规划
  - 项目后评估：复盘、经验教训库、根因分析（5 Whys、鱼骨图）
- 4. 标准与规范
  - ISO/IEC 12207：软件生命周期过程
  - ISO/IEC 29119：软件测试标准
  - ISO/IEC 25010：系统与软件质量模型
  - CMMI、SPICE（ISO 15504）
  - 国内标准：GB/T 8566（生命周期过程）、GB/T 8567（文档编制）
  - 安全合规：等保 2.0、OWASP Top 10、SDL 安全开发生命周期
- 5. 安全工程
  - OWASP Top 10 要点
    - 注入类（SQL / 命令 / LDAP 注入）、失效的身份认证
    - 敏感数据暴露、XML 外部实体 XXE、失效的访问控制
    - 安全配置错误、跨站脚本 XSS、不安全的反序列化
    - 使用含已知漏洞的组件、日志与监控不足
  - 认证与授权：OAuth 2.0、OIDC、JWT、Session、SSO、RBAC / ABAC
  - 传输与存储安全：TLS、证书管理、字段级加密、密钥管理 KMS
  - 安全开发：威胁建模 STRIDE、SDL 流程、代码审计、依赖漏洞扫描
  - 测试手段：SAST 静态扫描、DAST 动态扫描、IAST 插桩、模糊测试
  - 供应链安全：SBOM 软件物料清单、依赖投毒、签名验证
  - 合规与隐私：等级保护 2.0、数据安全法、个人信息保护法

四、应用领域

- 1. Web 与互联网应用
  - 特征：高并发、快速迭代、前后端分离
  - 技术：React / Vue、Spring Boot、Node.js、Nginx、Redis、MySQL
- 2. 移动应用
  - 原生：iOS（Swift / SwiftUI）、Android（Kotlin / Jetpack）
  - 跨端：Flutter、React Native、uni-app
  - 特征：包体积、耗电、弱网、热更新合规
- 3. 企业级应用
  - 类型：ERP、CRM、OA、HR、财务
  - 特征：流程复杂、权限体系、数据一致性、集成对接
- 4. 金融核心系统
  - 特征：强一致、高可用、可审计、监管合规
  - 技术：分布式事务、两地三中心、单元化、账务对账
- 5. 工业软件
  - 类型：CAD / CAE / CAM、MES、SCADA、PLC
  - 特征：实时性、可靠性、与物理设备耦合、国产化替代
- 6. 嵌入式与实时系统
  - 特征：资源受限、硬实时、中断驱动、安全认证（DO-178C、ISO 26262）
- 7. 游戏开发
  - 引擎：Unity、Unreal
  - 特征：渲染性能、网络同步、资源热更、反外挂
- 8. 云原生与 SaaS
  - 特征：多租户、弹性伸缩、按量计费、可观测性
  - 技术：Kubernetes、Service Mesh、Serverless、多租户隔离
- 9. 大数据与 AI 系统
  - 特征：数据管道、模型服务、特征工程、MLOps
  - 技术：Spark / Flink、模型服务（Triton / vLLM）、实验追踪
- 10. 开源软件与社区
  - 许可证：MIT、Apache-2.0、GPL、AGPL、BSD
  - 治理：贡献流程、CLA、版本发布、社区运营

五、工程实践与能力路径

- 1. 团队与协作
  - 研发流程：需求池 → 评审 → 排期 → 开发 → 测试 → 发布 → 复盘
  - 角色分工：产品经理 PM、业务分析 BA、架构师、开发、测试 QA、运维 Ops、SRE
  - 协作工具：Jira、Confluence、GitLab、飞书 / 钉钉、Slack
  - 协作机制：Code Review、结对编程、技术分享、故障复盘会
  - 沟通成本：康威定律 —— 系统架构会映射组织沟通结构
- 2. 工程实践
  - 需求到上线的全链路闭环
  - 文档体系：需求文档、设计文档、接口文档、部署文档、运维手册
  - 发布管理：版本号规范、发布窗口、回滚预案
  - 监控告警：指标、日志、追踪三位一体；告警分级与降噪
  - 故障处理：应急响应、止损优先、根因分析、改进项跟踪
  - 安全开发：SDL、依赖漏洞治理、密钥与凭据管理、最小权限
- 3. 能力模型
  - 计算机基础：数据结构与算法、操作系统、计算机网络、数据库原理
  - 编程能力：至少一门主力语言 + 一门脚本语言；掌握调试与性能分析
  - 工程能力：Git、Linux、测试、CI/CD、容器
  - 架构能力：设计模式、分布式、高并发、高可用、领域建模
  - 软技能：沟通表达、文档写作、项目管理、团队协作
- 4. 学习路径
  - 阶段 0：编程基础（1–2 个月）
    - 一门语言语法、调试、Git 基础
    - 验收：能独立完成命令行小工具并提交到仓库
  - 阶段 1：计算机基础（2–3 个月）
    - 数据结构与算法、操作系统、计算机网络、数据库
    - 验收：能手写常见数据结构，理解一次 HTTP 请求全链路
  - 阶段 2：应用开发（2–3 个月）
    - Web 或移动框架、数据库设计、接口开发
    - 验收：独立完成一个带前后端与数据库的完整项目
  - 阶段 3：软件工程理论（1–2 个月）
    - 需求、设计、UML、测试、项目管理
    - 验收：能为一个中等规模系统写出需求规格与设计文档
  - 阶段 4：工程化与实战
    - CI/CD、容器、云服务、性能优化、开源贡献、实习
- 5. 职业路径
  - 技术序列：初级开发 → 中级 → 高级 → 技术专家 / 架构师
  - 管理序列：技术经理 → 项目经理 → 研发总监 → CTO
  - 横向转型：测试开发、SRE、产品经理、技术顾问、解决方案架构师
- 6. 常见认知偏差（FAQ）
  - 「软件工程 = 写代码」：编码约占 20%，需求、设计、测试、维护占大头
  - 「敏捷 = 不写文档」：敏捷要的是可工作的软件，而非零文档
  - 「测试是测试岗的事」：质量内建，开发自测与自动化是前提
  - 「架构越复杂越专业」：合适优先，过度设计即负债
  - 「加班等于产出」：可持续节奏与工程质量才是长期主义
  - 「学会框架就会开发」：计算机基础决定职业天花板
  - 「重构是浪费时间」：技术债不还会以故障和延期形式连本带息偿还
- 7. 技术选型与架构治理
  - 选型维度：团队熟悉度、生态成熟度、性能、成本、可维护性、社区活跃度
  - 技术雷达：采纳（Adopt）、试验（Trial）、评估（Assess）、暂缓（Hold）
  - 架构治理：设计评审、架构守护测试（ArchUnit）、依赖约束、模块边界检查
  - 遗留系统处理：逆向工程、再工程、绞杀者模式（Strangler Fig）
  - 开源治理：许可证传染性（GPL/AGPL）、SBOM、供应链风险、贡献回馈
  - 文档与知识管理：ADR 决策记录、架构图即代码（C4 模型）、内部技术博客

Presented with xmind